

AI-Driven User Story Generation

Carlos Alberto dos Santos

Dép. d'informatique et de mathématique
Université du Québec à Chicoutimi
Chicoutimi, Canada
0000-0003-3824-6738

Kevin Bouchard

Dép. d'informatique et de mathématique
Université du Québec à Chicoutimi
Chicoutimi, Canada
0000-0002-5227-6602

Fabio Petrillo

Dép. de génie logiciel et des TI
École de Technologie Supérieure
Montreal, Canada
0000-0002-8355-1494

Abstract—User stories are pivotal in the realm of agile software development, primarily due to their organized structure and the simplicity with which they can be executed. Nevertheless, development teams encounter the arduous task of dealing with the assortment of information expected from diverse sources to manually write user stories. This undertaking not only consumes a significant amount of time but is also susceptible to inaccuracies. We automatically generated user stories relying on two distinct approaches: N-gram representation with linguistic heuristics and the GPT-3 model. We evaluated our work by using diverse user stories corpora and calculating the metrics BLEU, ROUGE-N, and BERTScore for each set of user stories generated by both approaches. For the N-gram approach we achieved an average: ROUGE-N=0.39, BLEU=0.26, BERTScore=0.73; and for GPT-3 model: ROUGE-N=0.46, BLEU=0.27 and BERTScore=0.69. We concluded that, although GPT models excel in producing more comprehensive user stories, N-gram models exhibit a higher level of semantic sensitivity. Considering its simplicity and minimal processing requirements, we recommend employing the N-gram technique for user story generation.

Index Terms—user story, text generation, large language model, n-gram, software engineering

I. INTRODUCTION

The adoption of agile processes assists software development teams in keeping the focus on the business needs, delivering software fit for purpose, and enabling software evolution. Moreover, these processes bring the practice of user story writing to represent usage scenarios in situations experienced by a system user [1].

In agile software development processes, each iteration encompasses a holistic approach to requirements engineering tasks, including elicitation, specification, documentation, and validation, all of which are handled collectively [2]. Software requirements must be unambiguous, testable and agreed upon with the customer, while a collection of requirements needs to be complete, consistent and feasible [3]. To achieve this result, development teams and customers must maintain robust communication and employ a common language to facilitate adjustments, thus guaranteeing the development of the correct software product. In this context, ASD brings the course of user story writing to represent scenarios of use in situations experienced by a system user [4].

Recent studies are tackling diverse aspects related to user stories involving *requirements elicitation and documentation*,

requirements analysis and negotiation, and *requirements validation and management* activities. However, only 25% of the studies related to user stories are concentrated on elicitation and documentation, with the most portion focused on using a given set of user stories in later Requirements Engineering activities [5]. It could indicate opportunities for more studies to support user stories specifications and requirements documentation.

In addition, the diverse array of data origins and textual configurations utilized as input for requirements engineering poses a challenge for software development teams [6]. They must review and grasp the client's requirements originating from various sources and transform them into user stories or any other format for specifying requirements. This process is usually carried out manually demanding significant time and effort by the team in each software development iteration. As an alternative to remedy this issue, there are approaches [7]–[9] that rely on automation to support user story elicitation and specification in ASD environments.

This paper presents two approaches to assist analysts in specifying context-based user stories: (i) a method combining N-gram frequency and linguistic heuristics; (ii) another method relying on a well-known *Large Language Model* (LLM) named *Generative Pre-trained Transformers* (GPT) to generate user stories with the support of a fine-tuning process to adapt it to our specific context. We evaluate our approaches using 22 different user stories corpora split between training and testing. We tested the accuracy of the models through the metrics ROUGE-N, BLEU and BERTScore [10]. Our results suggest that the N-gram model excels in the generation of user stories with a higher semantic score, though the GPT fine-tuned model could write more comprehensive user stories.

The automation of user story composition helps development teams speed up requirements refinement meetings and keep the focus on different angles related to the client's needs. Once the team has a list of potential user stories they can go further and discuss technical aspects, the development prioritization and the tasks assignment. The usage of N-gram models to assist the generation of user stories produces favourable results and proves to be efficient in terms of computational processing.

II. BACKGROUND

A. User story

Kent Beck, the creator of the Extreme Programming process, proposed the concept of story cards [11]. The cards were written using short sentences to describe the system's behaviour as a goal to incentivize the team discussion. All the cards were split into tasks to be completed by the development team. Later, Mike Cohn refined the definition of story cards [12]. He stated that story cards contain a short description of the user or customer-valued functionality, but the important parts are the conversations between stakeholders and developers about the story. He also described guidelines for writing good stories able to correctly guide the software development process.

User stories can be written by any development team member or stakeholder, in different phases of the project. They are usually based on conversations between the team members and stakeholders or requirements artifacts, such as project documents, standards, business rules or even emails. [12]. User stories employ a semi-structured template to express a functionality valuable for the user [13]. The usage of a template contributes to productivity and work deliverable quality [13]. Each user story has a goal to define an actor, the action, and the value delivered by this action in terms of business: *As a parent, I want to be able to connect with the staff in case of an emergency.* It brings the practice of keeping the focus on the client's perspective about the system and the business.

B. Text Prediction Models

Text prediction systems are used to estimate the next tokens or most probable tokens to occur in a sequence based on a text typed by the user [14]. Hamarashid *et al.* [15] and Garay-Vitoria *et al.* [14] presented important works in this area by summarizing all techniques used to build text prediction models and the aspects involved in constructing these systems. One of the approaches used in this experiment is N-gram which consists of a contiguous sequence of n items extracted from a provided text or speech sample. These items can encompass characters, words, or even larger units such as sentences. N-grams find application in various tasks, but when they are utilized for text prediction using only a single word to make predictions, it is referred to as a unigram model, when two words are used, a bigram, and so on. Jurafsky *et al.* [16] study had the greatest influence on this approach. They presented valuable insights and best practices to develop functional N-gram models.

These systems have become famous in the field of assistive technologies, being an important support to people with disabilities interacting with devices and other people. However, newer applications appeared, where these systems were used to facilitate the operation of devices by humans to perform daily tasks [15].

C. Large Language Model - LLM

The Large Language Model (LLM) undergoes training by processing extensive quantities of unlabeled text using a range of pretraining objectives. This process equips the model with the ability to autonomously learn from the text [17] generating its learning target (self-supervised learning). In a nutshell, a LLM can be used to generate text from the input of a random text. Much of what is known about large language models was introduced and formalized by two keystone papers: BERT [18] and GPT [19]. Both are trained in different ways and can also generate text differently.

Since then, numerous new models have emerged [20] to tackle text generation, offering a wide array of parameters and approaches. Many of these models allow the user to fine-tune using specific corpora and so teach the model to generate answers more context-sensitive. By using GPT is possible to draft documents, write computer code, translate languages, analyze texts and much more. For this research, we fine-tuned a version of Generative Pre-trained Transformer (GPT) [19] to generate user stories since it is one of the most popular LLMs.

D. Quantitative Metrics

Natural language generation has been a longstanding goal for AI researchers, and it recently saw a significant breakthrough in the past decade, primarily due to the resurgence of deep neural networks. While we are finally able to generate texts that are consistent and human-like, albeit imperfect, the metrics to evaluate them are still quite simple. One of the still widely used metrics, for instance, is *Bilingual Evaluation Understudy*, commonly referred to as BLEU [21]. BLEU quantifies the similarity between generated text and reference texts by counting overlapping N-grams. It assigns a score from 0 to 1, with higher scores denoting closer similarity to the reference. BLEU's enduring popularity is due to its simplicity and low computational cost.

A second metric, with numerous variations, is Recall-Oriented Understudy for Gisting Evaluation, or ROUGE [22]. ROUGE is primarily designed for evaluating the quality of text generated by measuring the overlap of N-grams between the generated texts and reference texts, focusing on recall rather than precision. Furthermore, ROUGE is more suitable for tasks where multiple reference summaries may exist. Compared to BLEU, ROUGE places greater importance on the comprehensiveness of the generated content. Nevertheless, ROUGE suffers from the same problem as BLEU; it cannot measure semantical similarities and is hence limited to syntactical matches.

A particularly noteworthy metric to consider is BERTScore [10], which assesses sentence similarity by calculating the sum of cosine similarities between the embeddings of the tokens in two sentences. BERTScore is valuable as it effectively tackles two prevalent issues often encountered with N-gram-based metrics. The first one is the semantic comparison. For instance, BLEU & ROUGE fail to recognize that the sentence ["The building is well assembled"] is more similar to ["The house is nicely constructed"] than ["The house of

horror”], but BERTScore would. Furthermore, BERTScore avoids the limitations of previous metrics in capturing distant relationships. One simple example is causality such as A causes B vs B causes A [10], especially in a long sequence.

III. RELATED WORK

Some authors have already proposed the automatic generation of user stories [7]–[9]. Overall they used a combination of different natural language techniques to identify requirements data in corpora and then compose user stories from it. The adoption of a user story template is common ground in these works, as it eases the data identification and the user stories’ writing.

Our proposed approaches to automatically generate user stories are based on the N-gram model and GPT framework and they were not proposed by any other authors. The usage of the N-gram model proves to be enough to generate parts of user stories without relying on extensive data training or computational processing. On the other hand, the adoption of the GPT fine-tuning process shows that we can rely on LLMs for this task due to their capacity to generate more comprehensive text.

IV. USER STORY GENERATION

We employed two different models to generate user stories. The first one is an N-gram model supported by linguistic heuristics, and the second one relies on a fine-tuned GPT model. The corpora used for this research were initially introduced by Dalpiaz *et al.* [23] in their study about terminological ambiguity in user stories (the other six corpora were not used in this experiment as they were not publicly available).

The size of each corpus varies between 51 and 138 user stories. A total of 10 sentences of each corpus was separated for the evaluation step described in the Session V. We separated them manually trying to get different sorts of statements to run our tests, focusing on different roles and actions. The rest of the sentences were used to build context-based models for each corpus of user stories. Thus, we built 22 N-gram models and fine-tuned 22 versions of the GPT model for this study. Each model contains only tokens presented in its user stories.

The following subsections present the details of our model implementation.

A. N-gram model

The proposed approach is based on a text generation system built using a statistical N-gram model. The choice for this method is based on the models’ evaluation study produced by Hamarashid *et al.* [15]. This statistical model was formulated utilizing a user stories corpora sample [23] to aid software professionals in user story writing.

Figure 1 depicts this process. We designed this pipeline using NodeJS and established persistence with MongoDB to support our experiments. Firstly, about 90% of the sentences of each corpus are added into a database collection using a seed operation. Secondly, every user story added to the collection automatically updates the N-gram model with new

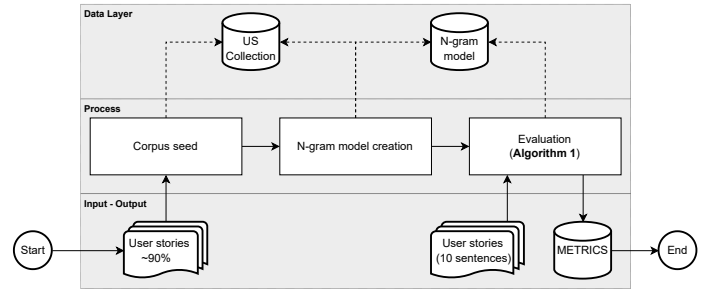


Fig. 1. Workflow diagram of the experimental protocol.

tokens and their frequencies. We used the Natural JS library¹ to separate the user stories into N-grams. Next, they are sorted by frequency and stored in a separate MongoDB collection using the following structure: *input* (N-gram), *output* (1-gram) and *count* (number of times the input and output appear on the text).

The N-gram collection works as a database of text completions. Any part of the user story can be completed with the content of this collection. Given a random input, as “*I want to*”, it is possible to find options to complete the sentence. The same works when the input is shorter or larger, as “*I*” or “*I want to have*”. The N-gram model created is calculated using $n=9$. We chose this number arbitrarily, but in the end, we noticed it was a good choice for creating this statistical model. The N-gram model can receive up to eight tokens as input and complete it with one token.

B. GPT model

The process to fine-tune a GPT model is well-described in the documentation² and there are several other tutorials available teaching the best practices for training and data preparation. There are numerous advantages in fine-tuning a GPT model, such as getting higher quality results than prompting and lower latency requests.

Currently, there are two different formats of data input to fine-tune a GPT model. The chat format and the text-generation legacy mode. We opted for the last one as we are not training a conversational chatbot, but a text generator. We have to design data for this procedure based on a key pair of *prompt* and *completion*. These are some examples of training data used to fine-tune our model:

```
"prompt": "As", "completion": " a"
"prompt": "I want", "completion": " to"
"prompt": "I want to", "completion": " be"
```

We fine-tuned the GPT model using as input unigrams, bigrams, and trigrams of our N-gram data collection. To do that, we created a JSON file with our completions and input

¹<https://naturalnode.github.io/natural/>

²<https://platform.openai.com/docs/introduction>

it into the OpenAI Platform using the model *davinci-002* as a base model. The training process takes around 30 minutes for each corpus to complete and it gives us a model ID as output that can be used to generate text via API or even directly on the OpenAI Playground.

V. EVALUATION

We simulated the generation of user stories using both proposed models. The idea behind this experiment is to mimic how the user could write a new user story being assisted by the models created. We believe that new user stories could be automatically added to both models to be reused in further elicitation sessions.

A. N-gram model

To evaluate the N-gram model, we coded a NodeJS function (1) that aimed to automate our analysis. This function has as an input the model ID (from 1 to 22) that we would like to perform the evaluation on. So, we clean the database and run the seed operation to create the user stories and N-gram collections in the MongoDB instance.

Algorithm 1: N-gram evaluation function

Data: Corpus Id
Result: USs Generated

```

1 remove all records from US collection;
2 remove all records from N-gram collection;
3 seed the model based on the Corpus Id;
4  $testUSs \leftarrow loadTestingUserStories(CorpusId)$ ;
5 foreach  $testUS \in testUSs$  do
6    $sentence \leftarrow [...]$ ;
7    $tokens \leftarrow tokenize(testUS)$ ;
8   append token to the sentence;
9   foreach  $token \in tokens$  do
10    append a token to the sentence;
11    if current token is not the last in tokens then
12      search for N-gram records matching the
        last eight tokens, ordered by count (desc),
        and limit to 5 (recursively until 1-gram);
13      if result NOT contains a record with an
        output equal to the next token then
14        search for 1-gram records matching the
        first character of the next token,
        ordered by count (desc), and limit to 5;
15        if result NOT contains a record with an
        output equal to the next token then
16          break out of the loop
17   append the sentence to output
18 return output
```

As a second step, we retrieve in memory the sample of testing user stories that we would like to write using the model. We split each user story into an array of tokens, and based on each sequence of tokens we try to find the next one using

the N-gram model. As our model was created using 9-grams, we can input into the model the top 8-grams to retrieve the most probable 9th-gram. This function works considering the best five options suggested by the model. It means that if the expected option is between the five first suggestions, it is selected and written in the sentence. If not, we still try to break the token into characters and use the first character of the token to retrieve the next token. If it is not found, we leave the function and present the sentence built as it is. We chose the cut-off of five suggestions arbitrarily, as we believe that the user will hardly scroll down far from five options to look for its desired token.

The output of the evaluation function is a set of user stories completely or partially written using the N-gram model proposed. For example, we tried to mimic the writing of the following sentence: *As a user I want to search and discover music based on other users similar to myself*. Our model was able to write this sentence partially using always the first five completing options suggested by our model: *As a user I want to search and discover music based on other*. Even partially, the model proved itself capable of writing most of the sentence, though in other rare cases, the model could complete only two tokens correctly.

We calculated the metrics BLEU, ROUGE-N, and BERTScore for all the sentences generated compared with the reference sentence. It gives us a quantitative result of our experiment. It helps us decide if we could proceed with a qualitative experiment with participants later to evaluate the feasibility of this approach. All the results reached on this experiment are available online on Zenodo³. Table I presents quantitative results for each corpus processed with the best and worst scores highlighted.

We calculated ROUGE-N considering $n=2$, or bigrams (two-word sequences) to capture more context and specific phrases. Doing this we can assess the presence of specific phrases and expressions in the generated user story. If using $n=1$ we could only capture the most basic and general information, overall text coherence and basic content overlap. The minimal score for ROUGE-N was 0.24, the maximum was 0.8, and the average was 0.39. For BLEU, the minimum was lower at 0.1 and the maximum stood at 0.75. BLEU's average score was lower at only 0.26. Finally, we can see that BERTScore was significantly higher with the lowest score at 0.65, the highest at 0.91, and the average at 0.73.

B. GPT model

For the GPT model, our evaluation was a bit more tricky. Trying to reproduce the same protocol as the N-gram model makes very little sense. Indeed, LLMs are better suited at generating complete answers or sentences than being used word by word like an autocomplete tool. Nevertheless, we did try it on a couple of sentences to check how it would perform. In that case, we would send the input to GPT and check if the next word was among the top 5 proposed by the model.

³<https://doi.org/10.5281/zenodo.10044766>

TABLE I.
QUANTITATIVE METRICS FOR N-GRAM MODEL.

Corpus	Rouge	Bleu	BERTScore
c1	0.43	0.32	0.73
c2	0.26	0.14	0.65
c3	0.29	0.12	0.69
c4	0.37	0.23	0.73
c5	0.39	0.26	0.73
c6	0.32	0.15	0.7
c7	0.34	0.2	0.72
c8	0.45	0.33	0.77
c9	0.27	0.1	0.69
c10	0.31	0.16	0.72
c11	0.36	0.25	0.68
c12	0.46	0.32	0.74
c13	0.31	0.18	0.7
c14	0.46	0.38	0.74
c15	0.24	0.11	0.65
c16	0.8	0.75	0.91
c17	0.53	0.45	0.79
c18	0.33	0.17	0.71
c19	0.49	0.38	0.78
c20	0.31	0.16	0.71
c21	0.32	0.17	0.72
c22	0.48	0.38	0.79
AVG	0.39	0.26	0.73
MIN	0.24	0.1	0.65
MAX	0.8	0.75	0.91

If it was not, the user story generation would be terminated. Perhaps unsurprisingly, this process did not yield good results. We used only the top 5 since our goal is to simulate a user's behaviour who we believe would be unlikely to check more than 5 options.

Considering that we could not do exactly the same experiments as with N-gram, we opted for a methodology more adapted to LLM. More specifically, we used the OpenAI Playground to generate user stories using our fine-tuned models. To compare, we simply asked the model of each corpus to generate a user story statement based on the following input pattern: "As a role". The role was selected based on the testing user stories dataset, so by doing that we could better guide the GPT model to write a sentence similar to those expected on the testing dataset.

Then, for each corpus, we stored the generated sentence beside the reference user stories in spreadsheets. For example, for the reference user story "As a user I want to be able to enter my zip code and get a list of nearby recycling facilities so that I can determine which ones I should consider", we generated the sentence "As a user I want to know the status of the recycling center so that I can know which centers are available around my area".

Similarly to the N-gram model, we calculated the ROUGE-N, BLEU, and BertScore metrics for each generated sentence and made them available on Zenodo⁴. The Table II shows the results we obtained.

As it is possible to note, the minimal score for ROUGE-N was 0.34, the maximum was 0.57, and the average was 0.46.

⁴<https://doi.org/10.5281/zenodo.10044766>

TABLE II.
QUANTITATIVE METRICS FOR GPT MODEL.

Corpus	Rouge	Bleu	BERTScore
c1	0.42	0.29	0.71
c2	0.52	0.34	0.70
c3	0.47	0.27	0.70
c4	0.48	0.29	0.71
c5	0.46	0.25	0.68
c6	0.47	0.25	0.67
c7	0.49	0.29	0.71
c8	0.57	0.31	0.70
c9	0.43	0.24	0.65
c10	0.51	0.27	0.69
c11	0.43	0.24	0.61
c12	0.48	0.24	0.68
c13	0.44	0.22	0.68
c14	0.50	0.27	0.70
c15	0.52	0.30	0.70
c16	0.47	0.31	0.71
c17	0.50	0.26	0.71
c18	0.34	0.18	0.66
c19	0.57	0.30	0.73
c20	0.38	0.20	0.68
c21	0.40	0.24	0.66
c22	0.35	0.28	0.71
AVG	0.46	0.27	0.69
MIN	0.34	0.18	0.61
MAX	0.57	0.34	0.73

For BLEU, the minimum was lower at 0.18 and the maximum reached 0.34. BLEU's average score was lower at only 0.27. Finally, we can see that BERTScore was significantly higher with the lowest score at 0.61, the highest at 0.73, and the average at 0.69.

VI. DISCUSSION

We selected ROUGE and BLEU metrics to compare the quality of generated user stories with the reference dataset at a syntactic level, focusing on evaluating the correctness of token sequences. Additionally, we incorporated the BERTScore metric to perform a semantic analysis of our generated user stories. This combination of three metrics enabled us to determine which technique excels in producing complete and semantically accurate user stories when compared to our reference dataset.

The results indicated that the GPT models slightly outperformed the N-gram model in terms of the statistical metrics ROUGE and BLEU. GPT models were able to generate more comprehensive user stories, contributing to their higher scores in these areas. However, when we considered the BERTScore, the N-gram model demonstrated better performance. We concluded that while GPT models excel in producing complete sentences, they do not reach the same level of semantic accuracy as the user stories generated by the N-gram model.

This may appear surprising, however, it is important to consider the nature of the task. User stories are typically structured and repetitive, which may not fully leverage the sophisticated generalization capabilities of a Large Language Model (LLM). LLMs excel at generating human-like language and semantic generalization, but for this particular task, the

simplicity of the N-gram model proved effective. It can be quickly trained on a small yet specialized corpus, making it an ideal choice for user story generation across different branches of software development.

In the future, conducting experiments with real software developers is essential to validate these results and gauge potential productivity gains. In this context, the N-gram model could serve as a background tool to assist software development teams in completing text when writing user stories.

Another avenue for validation involves training the N-gram model on different corpus patterns. Given that software requirements can originate from diverse sources in various formats, development teams must read, analyze, and create user stories based on client requirements. By extracting and storing N-grams from these corpora, we can aim to expedite the analysis process during meetings and ensure the creation of a more cohesive set of user stories.

We should also explore the optimal size of the N-gram model. As we expand the model to store more user stories, it might result in increased processing time or more false positive completions. To address this, we can enhance our N-gram selection based on the user story template. By employing text parsers and heuristics, we can break down the user story into parts (role, action, and benefit) and store more precise N-grams in the model. Furthermore, developing more specific user story corpora for individual project initiatives can help the model maintain accuracy based on the project's unique textual context.

VII. CONCLUSION

We conducted a series of experiments in which we employed both N-gram and fine-tuned GPT models to create context-based user stories. Our assessment of these generated user stories incorporated the evaluation metrics ROUGE, BLEU, and BERTScore, examining both their syntactical and semantic aspects. Our findings indicated that while GPT models excelled in generating more comprehensive user stories, N-gram models demonstrated a higher degree of semantic sensitivity (as reflected in the BERTScore). Given the minor discrepancy in syntactic analysis and the difference in computational demands between the techniques, we chose the N-gram model as the preferred standard to support development teams in user story composition.

We also explored potential avenues for future research. This includes the consideration of a controlled experiment aimed at assessing the N-gram model's utility in aiding software practitioners in real-world user story creation scenarios. Additionally, we discussed enhancements to the model, such as its adaptability to training on diverse data formats and varying sizes, all while maintaining its accuracy.

REFERENCES

- [1] I. Sommerville, *Software Engineering*, ser. Always learning. Pearson, 2016.
- [2] B. Ramesh, L. Cao, and R. Baskerville, "Agile requirements engineering practices and challenges: an empirical study," *Information Systems Journal*, vol. 20, no. 5, pp. 449–480, 2010.
- [3] P. Bourque and R. E. Fairley, Eds., *SWEBOK: Guide to the Software Engineering Body of Knowledge*, version 3.0 ed. Los Alamitos, CA: IEEE Computer Society, 2014. [Online]. Available: <http://www.swebok.org/>
- [4] I. Sommerville, *Software Engineering*, 10th ed., ser. Always learning. Boston, MA: Pearson, 2016.
- [5] A. R. Amna and G. Poels, "Systematic literature mapping of user story research," *IEEE Access*, vol. 10, pp. 51 723–51 746, 2022.
- [6] G. N. Aranda, A. Vizcaíno, and M. Piattini, "A framework to improve communication during the requirements elicitation process in gsd projects," *Requirements Engineering*, vol. 15, no. 4, pp. 397–417, Nov 2010.
- [7] P. Rodeghero, S. Jiang, A. Armaly, and C. McMillan, "Detecting user story information in developer-client conversations to generate extractive summaries," in *Proceedings - 2017 IEEE/ACM 39th International Conference on Software Engineering, ICSE 2017*, 2017, pp. 49–59.
- [8] I. K. Raharjana, D. Siahaan, and C. Fatichah, "User story extraction from online news for software requirements elicitation: A conceptual model," in *JCSSE 2019 - 16th International Joint Conference on Computer Science and Software Engineering: Knowledge Evolution Towards Singularity of Man-Machine Intelligence*, 2019, pp. 342–347.
- [9] F. J. Peña Veitía, L. Roldán, and M. Vegetti, "User stories identification in software's issues records using natural language processing," in *2020 IEEE Congreso Bienal de Argentina, ARGENCON 2020 - 2020 IEEE Biennial Congress of Argentina, ARGENCON 2020*, 2020.
- [10] T. Zhang*, V. Kishore*, F. Wu*, K. Q. Weinberger, and Y. Artzi, "BERTscore: Evaluating text generation with bert," in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=SkeHuCVFDr>
- [11] K. Beck and C. Andres, *Extreme Programming Explained: Embrace Change*, ser. XP Series. Pearson Education, 2004.
- [12] M. Cohn, *User Stories Applied: For Agile Software Development*. USA: Addison Wesley Longman Publishing Co., Inc., 2004.
- [13] F. Dalpiaz, I. van der Schalk, S. Brinkkemper, F. B. Aydemir, and G. Lucassen, "Detecting terminological ambiguity in user stories: Tool and experimentation," *Information and Software Technology*, vol. 110, pp. 3–16, 2019.
- [14] N. Garay-Vitoria and J. Abascal, "Text prediction systems: a survey," *Universal Access in the Information Society*, vol. 4, no. 3, pp. 188–203, 2006. [Online]. Available: <https://doi.org/10.1007/s10209-005-0005-9>
- [15] H. K. Hamarashid, S. A. Saeed, and T. A. Rashid, "A comprehensive review and evaluation on text predictive and entertainment systems," *Soft Computing*, vol. 26, no. 4, pp. 1541–1562, 2022. [Online]. Available: <https://doi.org/10.1007/s00500-021-06691-4>
- [16] D. Jurafsky and J. H. Martin, *Speech and language processing : an introduction to natural language processing, computational linguistics, and speech recognition*. Pearson Prentice Hall, 2009.
- [17] E. Webber, *Pretrain Vision and Large Language Models in Python: End-To-end Techniques for Building and Deploying Foundation Models on AWS*. United Kingdom: Packt Publishing, Limited, 2023.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.
- [19] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018.
- [20] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Comput. Surv.*, vol. 56, no. 2, sep 2023. [Online]. Available: <https://doi.org/10.1145/3605943>
- [21] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, Jul. 2002, pp. 311–318. [Online]. Available: <https://aclanthology.org/P02-1040>
- [22] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81.
- [23] F. Dalpiaz, I. van der Schalk, S. Brinkkemper, F. B. Aydemir, and G. Lucassen, "Detecting terminological ambiguity in user stories: Tool and experimentation," *Information and Software Technology*, vol. 110, pp. 3–16, 2019, cited By :33.